Music Genre Classification

Kaggle Competition — Messy Mashup

Roll No: 24f1002643 | IIT Madras | Jan 2026

1. Abstract

This report presents the work done for the Jan 2026 Deep Learning and Generative AI Kaggle competition, which tasked participants with classifying audio clips into one of ten music genres. The dataset, named Messy Mashup, contained separated audio stems (vocals, drums, bass, other) per song, which posed an interesting challenge for both feature engineering and model design. Three distinct deep learning architectures were developed and evaluated — a fine-tuned Audio Spectrogram Transformer (AST), a custom residual CNN, and a hybrid CNN+Bidirectional GRU model. All training was done on GPU using PyTorch with mixed-precision, and synthetic data augmentation was applied to improve robustness. The primary metric used for evaluation was Macro F1 Score. The AST based model outperformed the other two, achieving the highest validation F1 of **0.922**, while the CNN+GRU model showed surprisingly competitive results considering its far lower parameter count. Experiment tracking was done via Weights & Biases.

2. Introduction

2.1 Problem Statement

The competition required building a model that can identify the genre of an audio clip from the following ten classes: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. What made this slightly unusual compared to a standard classification task is that the training audio was not provided as raw mixed tracks — it was provided as separated stems. Each "song" had four component files: vocals.wav, drums.wav, bass.wav, and other.wav. The test set however consisted of mixed files, so any model had to generalize from stem-based training to mixed audio at inference time.

2.2 Project Objectives

The main objectives I set out to achieve were:

- Explore how pre-trained transformer models, specifically the Audio Spectrogram Transformer, can be fine-tuned effectively on a relatively small music genre dataset.
- Design a lightweight custom architecture (CNN-based) that can act as a strong baseline without requiring large compute.
- Understand how different augmentation strategies (SpecAugment, noise injection, synthetic mashup generation) affect model robustness.
- Compare the three approaches systematically on validation metrics.

2.3 Report Structure

Section 3 describes the dataset and preprocessing pipeline. Section 4 discusses the feature extraction strategy. Sections 5.1 through 5.3 detail each model architecture. Section 6 presents comparative analysis and results. Section 7 concludes with learnings and future directions.

3. Dataset & Preprocessing

3.1 Exploratory Data Analysis

Dataset Quality

- 1,000 tracks across 10 genres, all 4 stems present with zero missing or corrupted files
- All stems standardized: 30s duration, consistent sample rate, mono channel

Spectral Characteristics per Stem

- **Bass:** Most distinctive stem with mel bins (20–250 Hz), narrowest bandwidth, lowest spectral centroid
- **Drums:** Broadband energy, highest zero-crossing rate, sharp RMS transient spikes
- **Vocals:** Mid-high range (300 Hz–3 kHz), intermittent activity with clear silent gaps
- **Other:** Diffuse harmonic energy spread across the mid-range

Genre-Level Patterns

- **Metal & Rock:** Highest RMS and spectral rolloff across all stems
- **Classical & Jazz:** Sparsest energy, near-silent drum stems
- **Hiphop:** Dominant bass envelope relative to other genres

Noise Augmentation (ESC-50)

- Bass is the most robust to noise as they have minimal overlap with environmental sound frequencies
- Vocals are the most fragile
- Optimal augmentation window: **SNR 5–15 dB**

3.2 Data Preprocessing & Augmentation

Rather than using individual stems directly, during training a "synthetic mashup" was created on-the-fly by loading all four stems for a song, applying per-stem random gain (0.6–1.4), and summing them. This simulates a real mixed track and forces the model to learn from full song representations. Additionally, with a probability of 0.8, a random ESC-50 noise clip was mixed in at low amplitude to improve noise robustness.

SpecAugment was applied on the GPU after mel spectrogram computation — time masking and frequency masking were applied with configurable widths (TIME_MASK and FREQ_MASK parameters in the CNN notebook). The entire mel spectrogram computation pipeline was implemented as a PyTorch nn.Module to run on the GPU, which significantly reduced CPU bottlenecks during training.

4. Feature Extraction Strategy

4.1 Mel Spectrogram (CNN / CNN+GRU Models)

For the custom CNN-based models, log-mel spectrograms were computed using torchaudio on the GPU. The key parameters were: sample rate of 16,000 Hz, 128 mel bins, FFT window size of 1024 samples (64 ms), and a hop length of 512 samples (32 ms). With a 20-second clip this produces approximately 625 time frames. The output is a (1, 128, ~625) tensor which is suitable for 2D convolution. The spectrogram values were log-compressed and passed through a per-batch normalization.

4.2 AST-Compatible Spectrogram

The Audio Spectrogram Transformer has specific preprocessing requirements derived from the ASTFeatureExtractor in HuggingFace. For the AST model, an n_fft of 400 and a hop length of 313 were used, producing 1024 time frames for a 20-second clip. Crucially, the spectrogram was normalized using AST's own constants: mean = -4.2677393 and std = 4.5689974 (using the formula: normalized = (dB_spec - AST_MEAN) / (AST_STD * 2)). This matched the normalization used during the model's original AudioSet pretraining, which was important to ensure the pretrained weights remain meaningful.

4.3 Justification

Log-mel spectrograms are widely considered the most effective representation for audio classification tasks as they roughly approximate the human auditory system. The choice of 128 mel bins captures sufficient frequency resolution for genre discrimination (e.g., distinguishing classical orchestral harmonics from metal distortion). Using the GPU-based transform pipeline (as a custom nn.Module) rather than CPU-based feature extraction was a deliberate engineering choice to avoid DataLoader bottlenecks — all raw waveforms are loaded to RAM first, and spectrograms are computed in batches during the forward pass.

5. Modeling & Experimentation

5.1 Model 1: Fine-Tuned Audio Spectrogram Transformer (AST)

Model 1 fine-tuned a pretrained Audio Spectrogram Transformer (AST) from HuggingFace using a two-phase strategy: first training only the classification head (5 epochs, LR=5e-4), then unfreezing all layers with layer-wise LRs (base=1e-5, head=1e-4) for 8 more epochs. Both phases used cosine scheduling with warmup and mixed precision training.

5.2 Model 2: Custom Residual CNN

Model 2 is a custom Residual CNN trained from scratch on (1, 128, T) mel spectrograms, using three stride-2 residual blocks (pre-activation ResNet style) with channel progression 32→64→128→256, followed by global average pooling and a dropout classifier head, optimized with AdamW + CosineAnnealingLR.

5.3 Model 3: CNN + Bidirectional GRU

Model 3 extends Model 2 by appending a 2-layer Bidirectional GRU after the CNN front-end to capture longer-range temporal patterns (rhythm, harmonic progressions) alongside the CNN's local spectral features — trained with the same configuration as Model 2.

6. Performance & Comparative Analysis

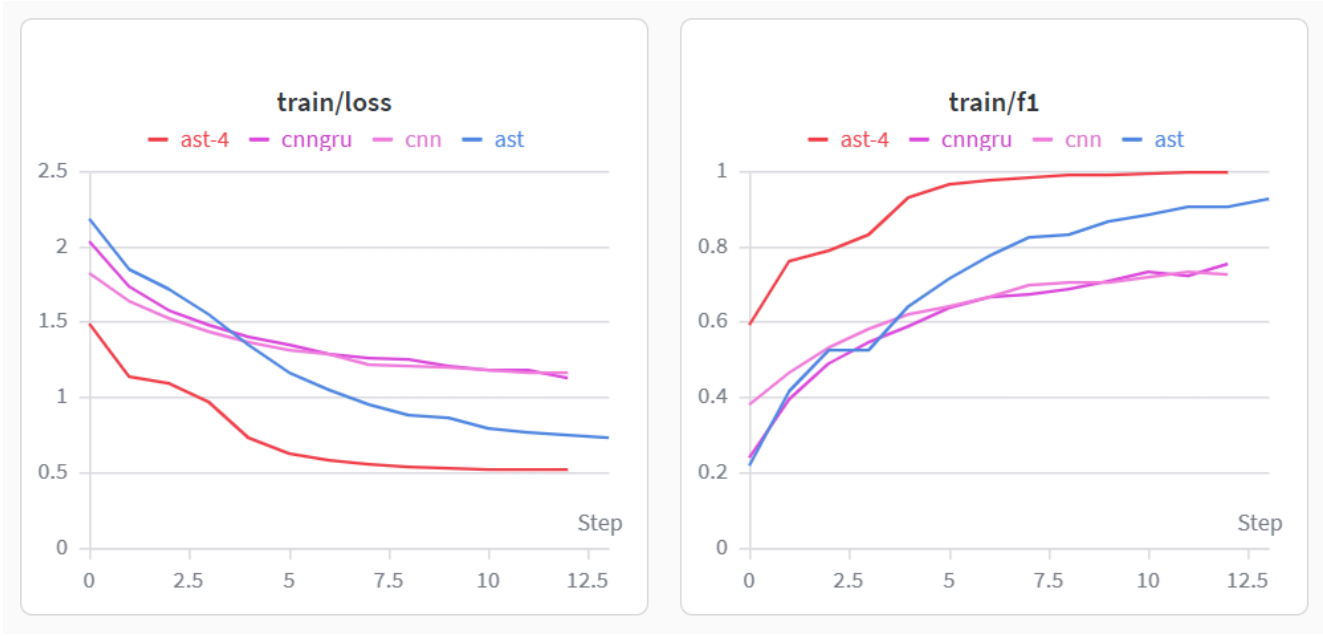
6.1 Evaluation Metrics

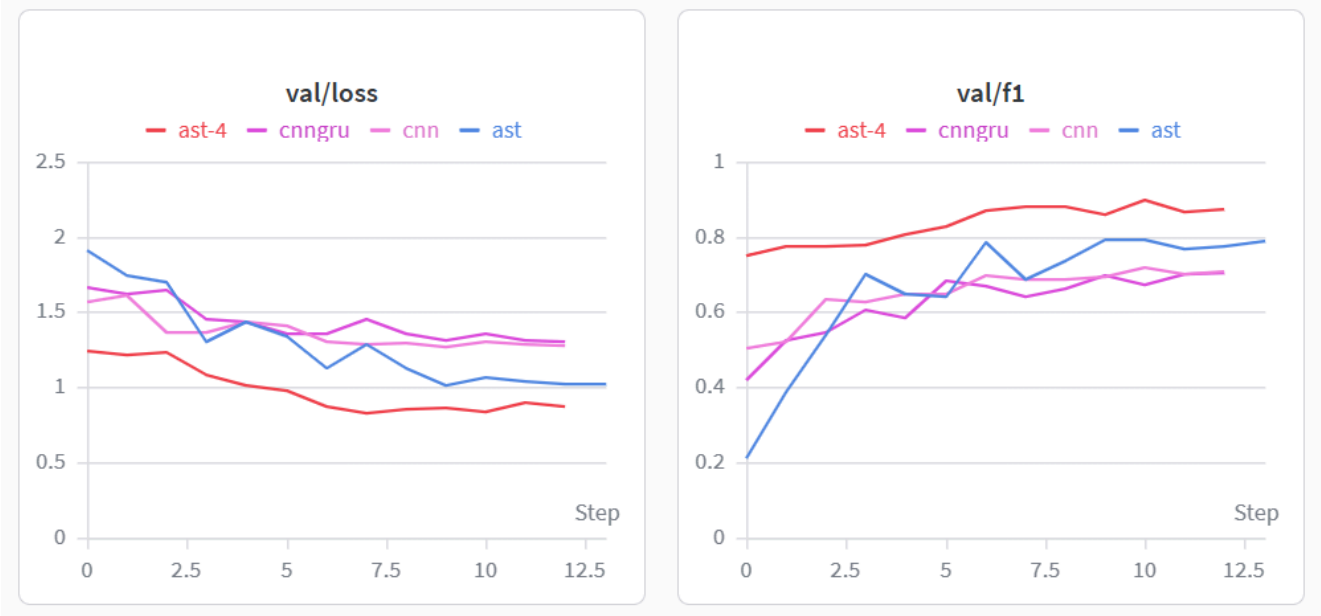
The primary evaluation metric, consistent with the Kaggle competition, was Macro F1 Score — an unweighted average of per-class F1 scores. This is appropriate for multi-class classification where class balance may not be perfect. Additional metrics tracked during training included cross-entropy loss (with label smoothing = 0.1) and per-epoch validation accuracy.

6.2 Training Details Summary

Parameter	AST	CNN	CNN+GRU
Epochs	3 (P1) + 10 (P2)	13	13
Optimizer	AdamW	AdamW	AdamW
Learning Rate	5e-4 / 2e-5	1e-3	1e-3
LR Scheduler	CosineAnnealingLR	CosineAnnealingLR	CosineAnnealingLR
Batch Size (eff.)	16	32	32
Parameters	~86M	~2M	~3M
Training Time	3 hrs	5 min	5 min

WandB Logs:





7. Conclusion & Future Work

7.1 Key Learnings

This project was quite insightful on several fronts. The most important takeaway was understanding how to adapt a large pretrained model (AST, originally trained on AudioSet) to a domain-specific task through careful two-phase fine-tuning. Aggressive fine-tuning from the start led to poor generalization, while the phased approach — head-only first, then full-model — produced more stable results.

Another learning was the value of the GPU-based preprocessing pipeline. Moving the mel spectrogram computation onto the GPU as part of the forward pass (instead of CPU-preprocessing in the DataLoader) essentially eliminated the data pipeline as a bottleneck and significantly improved GPU utilization.

7.2 Areas for Improvement

Several directions could push the score further:

- Ensemble the three models using soft voting on predicted probabilities.
- Fine-tune for more epochs.
- Explore more mixup augmentation at the waveform level

8. References

[1] Gong, Y., Liu, S., & Glass, J. (2021). AST: Audio Spectrogram Transformer. arXiv:2104.01778.

[2] Park, D. S., et al. (2019). SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. Interspeech 2019.

[3] HuggingFace Transformers library. https://huggingface.co/MIT/ast-finetuned-audioset-10-10-0.4593

[4] PyTorch torchaudio documentation. https://pytorch.org/audio/